Tunisian Republic
Ministry of Higher Education and Scientific Research
University of Carthage
National Institute of Applied Science and Technology

TP 2 : Intégration de la Sécurité dans un Pipeline CI/CD

Compte rendu — SAST, SCA et DAST automatisés dans GitHub Actions

Réalisé par :
Youssef Khalifa
Hanine Khemir
Oussema Said
Ahmed Amine Bououd

Classe : GL4 — Groupe 1

Dépôt : https://github.com/HanineKhemir/devsecops-tp2-cicd

Dépôt du TP1 (point de départ) : https://github.com/HanineKhemir/devsecops-tp1-sast-sca

Outils : GitHub Actions, Docker, pytest, Bandit (SAST), pip-audit / OSV (SCA), OWASP ZAP (DAST)

---

1. Objectif

Au TP1, nous avons trouvé et corrigé les failles de VulnBank en lançant les scanners **à la main**. Cette démarche a une limite évidente : elle dépend de quelqu'un qui pense à la faire. Rien n'empêche une faille de revenir dans le code au prochain commit.

Le TP2 transforme ces scans en **garde-fou automatique**. L'idée est de déplacer le contrôle de sécurité dans le pipeline d'intégration continue, à l'endroit où le code passe obligatoirement avant d'être fusionné :

	TP1 (manuel)	**TP2 (automatisé)**
Qui lance les scans	un étudiant, quand il y pense	le pipeline, à chaque push et chaque PR
Quand	une fois, en fin de TP	avant chaque fusion
Si une faille est trouvée	on la note dans un rapport	**la fusion est refusée**
Couverture	SAST + SCA	SAST + SCA + **DAST** + tests de non-régression
Nouvelle CVE sur une dépendance	personne ne la voit	détectée au run suivant

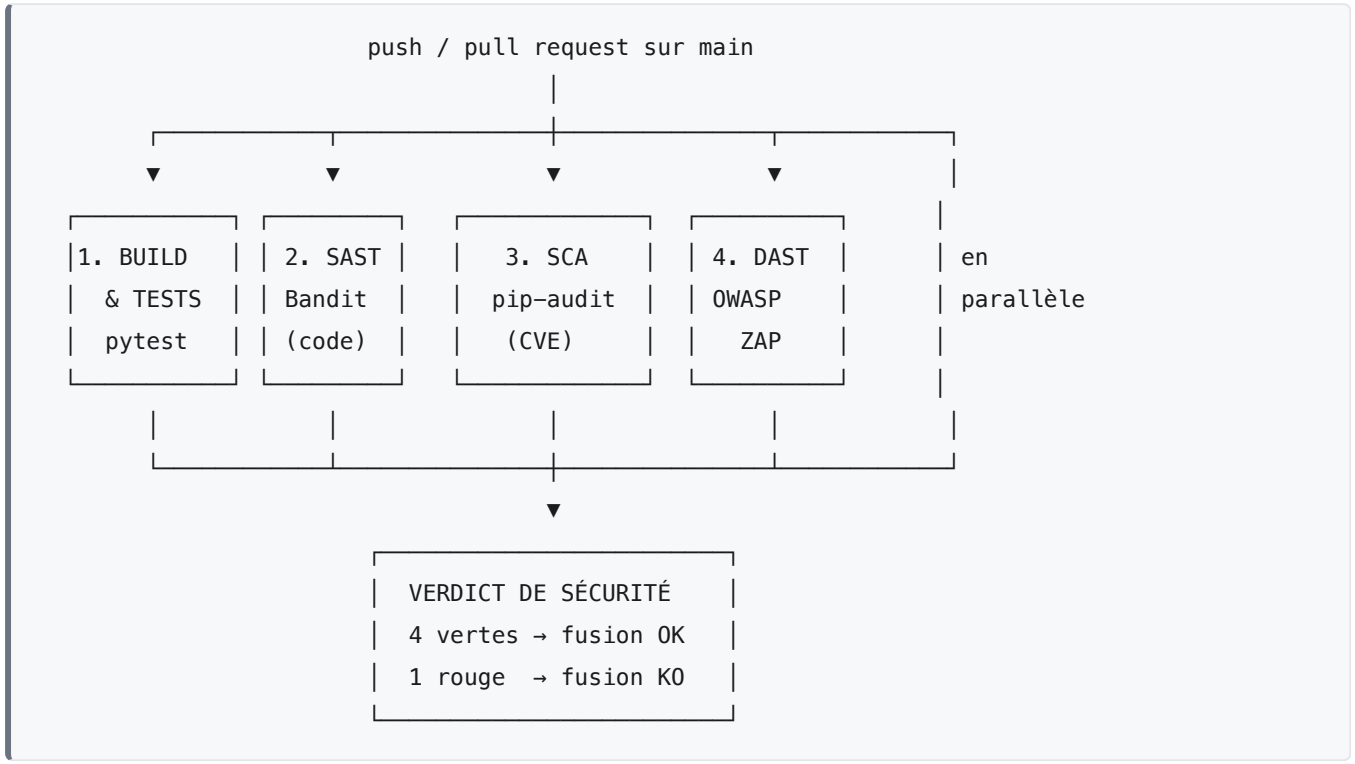
Le pipeline n'est donc plus seulement un outil de productivité : c'est un **contrôle d'accès au dépôt**.

---

2. L'application et son pipeline

L'application est **VulnBank**, reprise du TP1 dans son état corrigé : une application Flask avec base SQLite (connexion, consultation de comptes, commentaires, export, taux de change).

Le pipeline est décrit dans un seul fichier, `.github/workflows/ci-security.yml`, et comporte **quatre portes** plus un job de verdict :



#	Porte	Outil	Ce qu'elle regarde
1	Build & tests	pytest	le code compile, et 14 tests de non-régression de sécurité passent
2	**SAST**	Bandit	le **code source** (analyse statique, sans exécuter)
3	**SCA**	pip-audit / OSV	les **dépendances** (CVE connues)
4	**DAST**	OWASP ZAP	l'**application qui tourne** (vraies requêtes d'attaque)

Pourquoi les quatre jobs sont indépendants

Les quatre portes tournent **en parallèle**, sans dépendance entre elles. Ce choix a deux raisons :

1. **la vitesse** : le pipeline complet prend le temps de la porte la plus lente (~4 min) au lieu de la somme des quatre ;
2. **la pédagogie, et l'utilité réelle** : une PR vulnérable doit montrer **tous** ses problèmes d'un coup. Si les scanners dépendaient de la porte 1, des tests rouges les empêcheraient de s'exécuter et le développeur devrait corriger, repousser, attendre, découvrir une deuxième faille, et ainsi de suite.

Le job `security-gate` agrège les quatre résultats. C'est lui qui est déclaré **check requis** dans la protection de branche de `main` : tant qu'il est rouge, le bouton de fusion de GitHub est désactivé.

---

3. Les règles de blocage

C'est le cœur du TP : *qu'est-ce qui, exactement, doit arrêter le pipeline ?*

3.1 Pourquoi des scripts et pas le code de sortie des scanners

Un scanner sort en erreur dès qu'il trouve **quoi que ce soit**, même une remarque informative. Si on branche directement son code de sortie sur le pipeline, celui-ci devient rouge en permanence — et une équipe qui voit du rouge en permanence finit par ignorer le rouge. C'est le principal mode d'échec d'un garde-fou mal réglé.

Nous avons donc écrit une **porte** par scanner : un petit script Python, sans dépendance, qui lit le rapport JSON et décide.

Script	Porte	Règle de blocage
`scripts/gate_bandit.py`	SAST	bloque si ≥ 1 constat de sévérité **HIGH**
`scripts/gate_sca.py`	SCA	bloque si ≥ 1 **CVE connue** sur une dépendance
`scripts/gate_zap.py`	DAST	bloque si ≥ 1 alerte de risque **High** (`riskcode = 3`)

Trois propriétés communes, écrites dans `scripts/gate_common.py` :

- **la règle est lisible** et identique en local et en CI : un développeur peut rejouer exactement le verdict sur son poste avant de pousser ;
- **les exceptions sont documentées** : chaque script a un dictionnaire `EXCEPTIONS` (identifiant → justification + date). Un faux positif se traite par une ligne justifiée et relue en revue de code, pas en désactivant la porte ;
- **un rapport manquant = porte rouge**. Si le scanner plante, le script ne trouve pas son fichier JSON et bloque. Sinon il suffirait de casser le scanner pour passer la porte — un scanner muet n'est pas un scanner rassurant.

3.2 Le choix des seuils

SAST — HIGH seulement. Bandit classe en MEDIUM/LOW beaucoup de constats de style ou de contexte. Ses constats HIGH, eux, correspondent à des failles exploitables : `B605` (injection de commande), `B501` (certificat TLS non vérifié), `B201` (mode debug actif). C'est la traduction directe de « critique ou haute » pour cet outil.

SCA — politique stricte. La sortie JSON de pip-audit ne contient **pas** de score CVSS : elle donne l'identifiant de la faille, ses alias CVE et la version corrigée, mais pas la sévérité. Filtrer sur « HIGH/CRITICAL » n'est donc pas possible de façon fiable sans interroger une autre base. Nous bloquons donc sur **toute CVE connue** : c'est un **surensemble** de la règle demandée, et c'est tenable ici parce que nos versions sont épinglées et déjà corrigées. Quand une CVE ne nous concerne pas (faille dans une fonction non utilisée), le mécanisme d'exceptions la neutralise explicitement.

DAST — High seulement. Un scan ZAP renvoie toujours une série de MEDIUM d'en-têtes manquants (CSP, X-Frame-Options, Anti-CSRF). Ce sont des durcissements souhaitables, pas des failles exploitées. Les alertes High, elles, correspondent à une exploitation **effectivement réussie** par ZAP pendant le scan actif. Nous bloquons sur High et affichons les MEDIUM en avertissement.

---

4. L'environnement éphémère et le scan DAST

C'est la partie nouvelle par rapport au TP1 : pour faire du DAST, il faut une application **qui tourne**. Le job `dast` la déploie, la scanne, puis la détruit.

```
1. docker build                    → image vulnbank:ci
2. docker network create zapnet    → réseau privé du runner
3. docker run --network zapnet     → l'application, NON exposée au runner
4. attente du HEALTHCHECK          → on ne scanne pas une app pas démarrée
5. ZAP (zap-full-scan.py)          → spider + scan ACTIF
6. scripts/gate_zap.py             → verdict
7. docker rm -f / network rm       → destruction (if: always())
```

Trois points méritent d'être soulignés.

« Éphémère » veut dire jeté après usage. L'application naît au début du job et meurt à la fin, y compris si le scan échoue (`if: always()`). Chaque PR est donc testée sur une instance neuve, sans état hérité d'un run précédent, et il n'y a aucune infrastructure à maintenir entre deux scans.

Attendre la sonde de santé n'est pas un détail. Si ZAP démarre avant l'application, il ne trouve rien à attaquer : le rapport est vide, aucune alerte High, et **la porte passe au vert alors que rien n'a été testé**. C'est le pire cas possible pour un garde-fou — un faux négatif silencieux. Le job attend donc le `HEALTHCHECK` déclaré dans le `Dockerfile`, et échoue explicitement si l'application ne démarre pas.

Scan actif, pas scan passif. ZAP propose deux modes :

Mode	Ce qu'il fait	Trouve une injection SQL ?
`zap-baseline.py` (passif)	observe le trafic, lit les en-têtes	**non**
`zap-full-scan.py` (actif)	envoie de vraies charges d'attaque	**oui**

Le mode passif ne produit quasiment jamais d'alerte High : il ne verrait ni l'injection SQL ni le XSS réfléchi de notre démonstration. Nous utilisons donc le scan actif, avec l'option `-I` qui empêche ZAP de rendre lui-même le verdict — c'est notre script qui tranche.

L'environnement est rejouable à l'identique en local avec `docker-compose.ci.yml` :

```
docker compose -f docker-compose.ci.yml up --build --abort-on-container-exit
PYTHONPATH=scripts python scripts/gate_zap.py reports/zap.json
docker compose -f docker-compose.ci.yml down -v
```

---

5. Les tests de non-régression de sécurité

La première porte n'est pas un scanner : ce sont **14 tests pytest** qui rejouent les failles corrigées au TP1 (`tests/test_app.py`).

```
def test_injection_sql_dans_login_refusee(client):
    """admin' --  ouvrait la session sans mot de passe sur la version vulnérable."""
    reponse = client.post(
        "/login", data={"username": "admin' --", "password": "nimporte-quoi"}
    )
    assert reponse.status_code == 401
```

Test	Faille rejouée	Outil qui l'avait trouvée au TP1
`test_injection_sql_dans_login_refusee`	injection SQL (`admin' --`)	Bandit B608
`test_injection_sql_dans_comptes_ne_fuit_aucune_donnee`	injection SQL (`' OR '1'='1`)	Bandit B608
`test_xss_dans_commentaire_est_echappee`	XSS	**aucun scanner** — revue de code
`test_injection_commande_dans_export_refusee` (×4)	injection de commande	Bandit B605
`test_configuration_chargee_en_mode_sur`	`yaml.load()` non sûr	Bandit B506

Ces tests apportent deux choses que les scanners ne donnent pas :

1. ils vérifient le **comportement** (« cette saisie est-elle refusée ? ») et pas seulement la forme du code ;
2. ils couvrent le **XSS**, que ni Bandit ni SonarQube Community n'avaient signalé au TP1 — c'est précisément la faille que nous avions trouvée à la main. Elle est maintenant surveillée automatiquement.

Ils sont aussi la porte la **plus rapide** (3 secondes, sans Docker) : le développeur a son retour avant même que les scanners aient fini.

---

6. Validation : la preuve par deux pull requests

C'est la démonstration demandée par l'énoncé. Deux pull requests sur le même dépôt, visibles publiquement dans l'onglet *Pull requests* :

PR	Titre	Résultat
#1	`feat(search)` : recherche rapide (vulnérabilités intentionnelles)	**4 portes rouges — fusion bloquée**
#2	`fix(search)` : sécuriser la recherche rapide	**4 portes vertes — fusion autorisée**

6.1 PR #1 — la fonctionnalité écrite sans précaution

Nous avons ajouté une fonctionnalité réaliste : une route `GET /search?q=` pour retrouver un compte par nom de titulaire, avec son formulaire sur la page d'accueil. Écrite comme on l'écrit quand on va vite :

```
# vulnbank/app.py — version refusée par le pipeline
requete = (
    "SELECT owner, iban, balance FROM accounts "
    "WHERE owner LIKE '%" + terme + "%'"          # (1) concaténation SQL
)
lignes = conn.execute(requete).fetchall()

page = "<h2>Resultats pour " + terme + "</h2><ul>"
...
return render_template_string(page)               # (2) saisie rendue comme template
```

Et trois autres écarts du même genre : `os.system()` pour l'export (plus rapide que `iterdump()`), `debug=True` pour voir les erreurs, et un retour de `urllib3` en 1.26.5 (« la 2.x casse notre client HTTP »).

Verdict du pipeline : les quatre portes rouges.

Porte	Verdict	Ce qu'elle a trouvé
1. Build & tests	❌	**4 tests en échec** : `test_injection_commande_dans_export_refusee` (les 4 charges utiles : `x; rm -rf /`, `a && whoami`, `../../etc/passwd`, `$(id)`)
2. SAST (Bandit)	❌	**2 constats HIGH** : `B605` (`os.system` avec une saisie, `app.py:166`), `B201` (`debug=True`, `app.py:203`) — plus 1 MEDIUM `B608` (concaténation SQL)
3. SCA (pip-audit)	❌	**10 CVE** sur `urllib3 1.26.5`, dont `CVE-2023-45803`, `CVE-2024-37891`, `CVE-2025-50181`
4. DAST (OWASP ZAP)	❌	**5 alertes High**, toutes sur le paramètre `q` de `/search`

Les cinq alertes High de ZAP, avec la charge utile qui a réellement fonctionné :

Alerte ZAP	CWE	Charge utile envoyée par ZAP
`SQL Injection`	CWE-89	`/search?q='` → HTTP 500, la requête SQL est cassée par l'apostrophe
`Cross Site Scripting (Reflected)`	CWE-79	`</h2><scrIpt>alert(1);</scRipt><h2>` — retrouvé tel quel dans la réponse
`Cross Site Scripting (DOM Based)`	CWE-79	`<script>alert(5397)</script>`
`Server Side Template Injection`	CWE-1336	`zj{{2756*1993}}zj` — **ZAP a récupéré le résultat du calcul**
`Source Code Disclosure - File Inclusion`	CWE-541	`/search?q=ZAP`

La découverte la plus instructive du TP est cette quatrième ligne. Nous avions introduit `render_template_string()` en pensant créer un XSS. ZAP a montré que c'est bien plus grave : la saisie n'est pas seulement affichée, elle est **évaluée comme du code Jinja2 sur le serveur**. En envoyant

`{{2756*1993}}`, ZAP a reçu le produit en retour — la preuve que du code arbitraire s'exécute côté serveur (SSTI, chemin classique vers l'exécution de commandes à distance).

Et surtout :

> **Bandit n'avait classé la concaténation SQL qu'en MEDIUM (non bloquant) et n'avait rien dit du XSS ni du SSTI. Sans la porte DAST, ces trois failles auraient franchi le pipeline.**

C'est la justification concrète du TP : le SAST et le DAST ne voient pas la même chose. Le premier lit le code et raisonne sur sa forme ; le second attaque l'application et observe ce qui se passe vraiment.

Côté GitHub, la conséquence est mécanique : le job `security-gate` étant déclaré **check requis** sur `main`, l'état de la PR passe à `BLOCKED` et le bouton de fusion est désactivé. Ce n'est pas un avertissement que l'on peut ignorer.

6.2 PR #2 — la même fonctionnalité, écrite correctement

La correction **garde la fonctionnalité** : `/search` existe toujours, avec son formulaire. Seules les failles disparaissent.

```
# vulnbank/app.py — version acceptée par le pipeline
lignes = conn.execute(
    "SELECT owner, iban, balance FROM accounts WHERE owner LIKE ?",
    (f"%{terme}%",),                    # (1) paramètre lié, jamais du code SQL
).fetchall()

return render_template("search.html", terme=terme, resultats=lignes)
                                        # (2) vrai template -> échappement Jinja2
```

Vulnérabilité	Correction
Injection SQL	requête paramétrée (`?`) : la saisie est une donnée, pas du code
XSS + SSTI	rendu par `templates/search.html` ; Jinja2 échappe les variables
Injection de commande	retour à la liste blanche (regex) + API SQLite `iterdump()`, sans shell
Mode debug	`debug` lu dans `FLASK_DEBUG`, désactivé par défaut
`urllib3 1.26.5`	retour en `urllib3 2.8.0`

Verdict du pipeline : les quatre portes vertes.

Porte	Verdict	Détail
1. Build & tests	✅	**17 tests** passants (14 + 3 ajoutés pour `/search`)
2. SAST (Bandit)	✅	0 HIGH, 0 MEDIUM, 0 LOW sur 132 lignes
3. SCA (pip-audit)	✅	0 CVE sur 13 dépendances
4. DAST (OWASP ZAP)	✅	**0 High**, 4 Medium, 6 Low, 3 Info — 50 requêtes d'attaque sur 7 routes, `/search` incluse

L'état de la PR passe à `CLEAN` : la fusion est autorisée.

Un point de méthode : **la correction est accompagnée de ses tests**. Trois tests de non-régression ont été ajoutés sur `/search` (injection SQL, XSS/template, et une contre-épreuve vérifiant que la recherche légitime fonctionne toujours). La porte DAST a trouvé les failles en 3 minutes ; les tests unitaires les verrouillent désormais en 3 secondes, à chaque run.

6.3 Vérifier que le vert n'est pas un faux vert

Une porte verte n'a de valeur que si le scan a réellement eu lieu. Nous avons donc vérifié les rapports publiés en artifacts, et non seulement la couleur du job :

- le rapport ZAP de la PR #2 contient **13 alertes et 50 instances testées**, et la route `/search` figure bien dans les URL explorées : le scan a donc eu lieu, et c'est bien l'absence d'alerte High qui rend la porte verte ;
- à l'inverse, un rapport **vide** signifierait que ZAP n'a rien trouvé à attaquer (application pas démarrée) ; c'est précisément le faux négatif que l'attente du `HEALTHCHECK` évite ;
- un rapport **manquant** fait échouer la porte, par construction (`scripts/gate_common.py`).

6.4 Extraits des journaux du pipeline

Les sorties ci-dessous sont copiées telles quelles depuis les runs GitHub Actions des deux pull requests. Elles montrent le format de nos portes : le constat, l'endroit exact, et le verdict.

PR #1 — porte 1 (tests unitaires)

```
========================== short test summary info ===========================
FAILED tests/test_app.py::test_injection_commande_dans_export_refusee[x; rm -rf /] - assert 200
 +  where 200 = <WrapperTestResponse streamed [200 OK]>.status_code
FAILED tests/test_app.py::test_injection_commande_dans_export_refusee[a && whoami] - assert 200
 +  where 200 = <WrapperTestResponse streamed [200 OK]>.status_code
FAILED tests/test_app.py::test_injection_commande_dans_export_refusee[../../etc/passwd] - asser
 +  where 200 = <WrapperTestResponse streamed [200 OK]>.status_code
FAILED tests/test_app.py::test_injection_commande_dans_export_refusee[$(id)] - assert 200 == 40
 +  where 200 = <WrapperTestResponse streamed [200 OK]>.status_code
======================== 4 failed, 10 passed in 4.09s ========================
```

PR #1 — porte 2 (SAST / Bandit)

```
                PORTE SAST — Bandit (analyse statique du code)
================================================================================
Lignes de code analysees : 153
Constats : 2 HIGH, 1 MEDIUM, 0 LOW

[BLOQUANT] B605 — start_process_with_a_shell
  Fichier    : vulnbank/app.py:166
  Probleme   : Starting a process with a shell, possible injection detected, security issue.
  Confiance  : HIGH
  CWE        : CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
  Code       : 165     # Export rapide en appelant directement l'outil sqlite3.

[BLOQUANT] B201 — flask_debug_true
  Fichier    : vulnbank/app.py:203
  Probleme   : A Flask app appears to be run with debug=True, which exposes the Werkzeug debugg
  Confiance  : MEDIUM
  CWE        : CWE-94 (https://cwe.mitre.org/data/definitions/94.html)
  Code       : 202     host = os.environ.get("HOST", "127.0.0.1")

[AVERTISSEMENT] B608 (vulnbank/app.py:115) — Possible SQL injection vector through string-based
--------------------------------------------------------------------------------
PORTE SAST : ROUGE — 2 probleme(s) bloquant(s).
```

PR #1 — porte 3 (SCA / pip-audit), extrait des 10 CVE :

```
            PORTE SCA — pip-audit / OSV (securite des dependances)
================================================================================
Dependances analysees : 13
Dependances vulnerables : 1
Vulnerabilites trouvees : 10

[BLOQUANT] urllib3 1.26.5
  Identifiant      : PYSEC-2023-192
  Alias (CVE)      : CVE-2023-43804, GHSA-v845-jxx5-vc9f
  Versions corrigees : 1.26.17, 2.0.6
  Description      : urllib3 is a user-friendly HTTP client library for Python. urllib3 doesn't

[BLOQUANT] urllib3 1.26.5
  Identifiant      : PYSEC-2023-212
  Alias (CVE)      : GHSA-g4mx-q9vg-27p4, CVE-2023-45803
  Versions corrigees : 1.26.18, 2.0.7
  Description      : urllib3 is a user-friendly HTTP client library for Python. urllib3 previou
```

PR #1 — porte 4 (DAST / OWASP ZAP) : les charges utiles qui ont fonctionné.

```
         PORTE DAST — OWASP ZAP (scan actif de l'application lancee)
================================================================================
Version de ZAP : 2.17.0
Cible scannee  : http://vulnbank:5000
Alertes : 5 High, 4 Medium, 6 Low, 3 Info

[BLOQUANT] Cross Site Scripting (DOM Based) (risque High)
  Site      : http://vulnbank:5000
  CWE       : CWE-79
  Confiance : 3
  -> GET http://vulnbank:5000/search?q=%3Cscript%3Ealert(5397)%3C/script%3E
     parametre vulnerable : q
     charge utile         : <script>alert(5397)</script>

[BLOQUANT] Cross Site Scripting (Reflected) (risque High)
  Site      : http://vulnbank:5000
  CWE       : CWE-79
  Confiance : 2
  -> GET http://vulnbank:5000/search?q=%3C%2Fh2%3E%3CscrIpt%3Ealert%281%29%3B%3C%2FscRipt%3E%3C
     parametre vulnerable : q
     charge utile         : </h2><scrIpt>alert(1);</scRipt><h2>
     preuve dans la reponse : </h2><scrIpt>alert(1);</scRipt><h2>

[BLOQUANT] SQL Injection (risque High)
  Site      : http://vulnbank:5000
  CWE       : CWE-89
  Confiance : 1
  -> GET http://vulnbank:5000/search?q=%27
     parametre vulnerable : q
     charge utile         : '
     preuve dans la reponse : HTTP/1.1 500 INTERNAL SERVER ERROR
```

PR #2 — porte 4 (DAST) après correction, sur la même route :

```
        PORTE DAST — OWASP ZAP (scan actif de l'application lancee)
================================================================================
Version de ZAP : 2.17.0
Cible scannee  : http://vulnbank:5000
Alertes : 0 High, 4 Medium, 6 Low, 3 Info

[AVERTISSEMENT] Absence of Anti-CSRF Tokens (Medium) — http://vulnbank:5000
[AVERTISSEMENT] Anti-CSRF Tokens Check (Medium) — http://vulnbank:5000
[AVERTISSEMENT] Content Security Policy (CSP) Header Not Set (Medium) — http://vulnbank:5000
[AVERTISSEMENT] Missing Anti-clickjacking Header (Medium) — http://vulnbank:5000
--------------------------------------------------------------------------------
PORTE DAST : VERTE — aucun probleme bloquant.
```

Le job de verdict agrège les quatre portes et affiche le tableau de synthèse sur la page du run :

```
PR #1 :  PIPELINE ROUGE — fusion bloquee. Voir les rapports en artifacts.
PR #2 :  PIPELINE VERT — fusion autorisee.
```

---

7. Limites et difficultés rencontrées

Un pipeline vert ne veut pas dire « application sûre ». Nos quatre portes sont vertes sur `main`, et pourtant ZAP signale encore 4 alertes MEDIUM (absence de jetons anti-CSRF, pas d'en-tête CSP, pas de protection contre le *clickjacking*). Nous avons choisi de ne pas bloquer sur ces points, mais ce sont de vraies faiblesses. Un pipeline définit un **plancher**, pas un objectif.

Le XSS reste un angle mort de l'analyse statique. Au TP1 déjà, ni Bandit ni SonarQube Community n'avaient vu le `render_template_string`. Dans la PR vulnérable, Bandit n'a classé la concaténation SQL qu'en MEDIUM et n'a rien dit du XSS : **sans la porte DAST, la PR serait passée sur ces deux failles**. C'est l'argument le plus net en faveur de la combinaison SAST + DAST : les deux outils ne voient pas la même chose.

La politique SCA stricte est un compromis. Bloquer sur toute CVE est rigoureux, mais une CVE publiée sur une dépendance populaire peut bloquer le dépôt entier du jour au lendemain, sans que notre code soit concerné. C'est exactement ce qui est arrivé avec `certifi` pendant ce TP. La réponse n'est pas d'abaisser la règle mais d'avoir un mécanisme d'exception rapide et tracé — d'où le dictionnaire `EXCEPTIONS` versionné dans le dépôt.

Le scan actif coûte du temps. La porte DAST prend à elle seule ~2 à 3 minutes (téléchargement de l'image ZAP, démarrage, spider, scan actif), contre quelques secondes pour les tests. Sur un vrai projet, on garde le scan actif sur les PR et on réserve les scans plus lourds (ZAP en mode authentifié, fuzzing) à une exécution nocturne.

Les secrets. L'application lit sa `SECRET_KEY` dans une variable d'environnement (correction du TP1, Bandit B105). Dans le pipeline, elle est injectée par `secrets.CI_SECRET_KEY`, avec `github.run_id` comme valeur de repli : l'environnement éphémère a donc une clé différente à chaque run, et aucun secret n'apparaît dans le dépôt.

Difficulté rencontrée. Le premier run du pipeline a échoué pour une raison qui n'avait rien à voir avec la sécurité : `ModuleNotFoundError: No module named 'vulnbank'`. En local nous lancions `python -m pytest` (qui ajoute le dossier courant à `sys.path`), alors que le pipeline lance `pytest` (qui ne le fait pas). Nous avons corrigé en explicitant `pythonpath = .` dans `pytest.ini` plutôt qu'en adaptant la commande : la configuration doit être dans le dépôt, pas dans la façon de l'invoquer. C'est une bonne illustration du fait qu'un pipeline ne reproduit jamais exactement le poste du développeur.

---

8. Conclusion

Le TP1 nous avait appris à **trouver** des failles. Le TP2 nous apprend à **ne plus les laisser entrer**.

Ce qui change concrètement :

- les scans ne dépendent plus de la vigilance de quelqu'un : ils sont déclenchés par le dépôt lui-même, à chaque pull request ;
- la décision est **écrite** : trois règles, trois scripts, lisibles et rejouables en local ;
- la décision est **appliquée** : la protection de branche de `main` fait du job `security-gate` un check requis, donc un pipeline rouge désactive réellement le bouton de fusion. Le garde-fou n'est pas un avertissement, c'est un refus ;
- le périmètre s'élargit : le DAST a trouvé deux failles (injection SQL et XSS réfléchi) que l'analyse statique n'aurait pas bloquées ;
- le temps joue pour nous : la CVE `certifi` est apparue **après** le TP1, sur un code inchangé. Le pipeline l'a signalée au premier run.

La démonstration par les deux pull requests le résume : la même fonctionnalité, écrite sans précaution, est refusée par quatre portes ; écrite correctement, elle est acceptée en quelques minutes. Le coût de la sécurité devient le coût d'une relecture, et il est payé **avant** la mise en production, pas après.

Annexe — Rejouer le pipeline en local

```
git clone https://github.com/HanineKhemir/devsecops-tp2-cicd.git
cd devsecops-tp2-cicd
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt -r requirements-dev.txt

# Porte 1 — tests unitaires
pytest

# Porte 2 — SAST
bandit -r vulnbank -f json -o reports/bandit.json --exit-zero
PYTHONPATH=scripts python scripts/gate_bandit.py reports/bandit.json

# Porte 3 — SCA
pip-audit -r requirements.txt -f json -o reports/pip-audit.json || true
PYTHONPATH=scripts python scripts/gate_sca.py reports/pip-audit.json

# Porte 4 — DAST (environnement éphémère : application + ZAP)
docker compose -f docker-compose.ci.yml up --build --abort-on-container-exit
PYTHONPATH=scripts python scripts/gate_zap.py reports/zap.json
docker compose -f docker-compose.ci.yml down -v
```

Glossaire

Terme	Définition
CI/CD	*Continuous Integration* / *Continuous Deployment* — chaîne automatique qui compile, teste et déploie le code à chaque modification.
SAST	*Static Application Security Testing* — analyse du code source, sans l'exécuter.
SCA	*Software Composition Analysis* — recherche de CVE connues dans les dépendances.
DAST	*Dynamic Application Security Testing* — attaque de l'application pendant qu'elle tourne.
CVE	*Common Vulnerabilities and Exposures* — identifiant public d'une vulnérabilité connue.
Porte (quality gate)	règle qui décide si le pipeline continue ou s'arrête.
Environnement éphémère	déploiement créé pour la durée d'un test, puis détruit.
Check requis	vérification que GitHub exige au vert avant d'autoriser une fusion.
Faux positif	alerte signalée par un outil, mais qui ne correspond pas à un risque réel.
Faux négatif	faille réelle qu'un outil n'a pas signalée — le cas le plus dangereux.
Shift-left	déplacer les contrôles de sécurité au plus tôt dans le cycle de développement.